

Product Requirements Document (PRD)

Gaurav Nandhan BA Studio

Enterprise Edition (v1.0.0)

Document Standard: BABOK v3 Compliant

Classification: Highly Confidential (Internal Enterprise Use Only)

Target Architecture: Next.js (Frontend), Vercel (Edge), Neon (PostgreSQL)

AI Engine: Google DeepMind Gemini 2.5 Pro / Flash

Last Updated: May 2026

1.0 Executive Summary & Introduction

1.1 Purpose of the Document

This Product Requirements Document (PRD) provides a comprehensive, exhaustive blueprint for the Gaurav Nandhan BA Studio. It is designed to serve as the single source of truth for software engineers, quality assurance (QA) testers, business analysts, and enterprise security officers. This document dictates the exact architectural requirements, LLM prompt engineering constraints, database schemas, and data flow pipelines required to maintain the enterprise integrity of the platform.

1.2 Product Vision & Objective

The core objective of the Gaurav Nandhan BA Studio is to autonomously transform unstructured, ambiguous meeting minutes (MOM) into a full suite of highly structured, BABOK v3-compliant technical documentation, interactive prototypes, and architectural diagrams. By deploying a Multi-Agent AI architecture powered by Gemini 2.5 Pro, the studio aims to reduce the traditional Business Analysis documentation lifecycle from weeks to a matter of minutes (achieving an 80%+ reduction in time-to-market for initial requirements gathering) while strictly maintaining enterprise security, logic validation, and traceability.

1.3 Scope of the System

This PRD outlines the requirements for the core system architecture, including:

- The Next.js 16 frontend rendering engine.
- The Vercel Serverless backend API routes.
- The NextAuth.js based Multi-Tenant Role-Based Access Control (RBAC) system.
- The `pii.ts` AI Shield data redaction middleware mechanism.

- The 11 integrated document generation modules (BRD, FRD, PRD, SRD, Pitch, UML, Flowcharts, Wireframes, Prototypes, Test Cases, Regulatory Audit).
- The Infrastructure-as-Code (Terraform) and End-to-End Automation (Playwright) manifest generators.

2.0 Business Requirements & Market Justification

2.1 Problem Statement

Traditional Business Analysis relies heavily on manual translation of stakeholder meetings into Business Requirements Documents (BRDs), Functional Requirements Documents (FRDs), and technical design artifacts. This analog process is highly prone to human error, cognitive bias, and extreme latency. A typical enterprise project requires 3-4 weeks of documentation before a single line of code is written, costing organizations thousands of dollars in lost productivity and misaligned requirements.

Furthermore, attempting to solve this problem by pasting meeting notes into public AI tools (like ChatGPT or public Gemini interfaces) poses massive data security risks, resulting in PII leaks and corporate data being used to train third-party models.

2.2 Proposed Solution

The Gaurav Nandhan BA Studio resolves both latency and security issues by providing a closed-loop, SOC2-compliant environment where users can safely ingest meeting notes. The proprietary "AI Shield" sanitizes the data before it touches an LLM. The system then utilizes a proprietary "Sovereign Versioning Engine" to parse the data and dispatch it to 11 distinct, highly-tuned AI Agents, each responsible for rendering a specific artifact in a standardized format.

2.3 Business Value (MoSCoW Prioritization)

Must Have (Critical Path for Enterprise MVP)

- **Multi-Agent Generation Engine:** The system must utilize distinct, specialized LLM agents (e.g., UX Architect, Senior BA, Systems Analyst) rather than a single monolithic prompt, to guarantee artifact specificity.
- **AI Shield (PII Redaction):** Automatic, immediate sanitization of all sensitive data (SSN, Emails, IPs) via Regex before transmitting to external LLMs.
- **Immutable Audit Logging:** Secure logging of all document generation and login actions, mapped to a specific user and organization.
- **Multi-Tenant RBAC:** Strict Role-Based Access Control ensuring `VIEWER` roles cannot consume enterprise billing quotas by generating documents.

Should Have (High Priority Enhancements)

- **Sovereign Versioning Engine:** A delta-analysis module that flags dependencies when new scope is added, explicitly marking impacted documents as "Stale/Review Required."
- **E2E Automation Suite Generation:** Generation of Playwright test scripts based on the AI-generated Wireframes or Prototypes to shift QA testing left.
- **Infrastructure as Code (IaC):** Generation of robust Terraform deployment manifests mapping to identified system requirements.
- **Jira Synchronization:** OAuth integration to automatically convert FRDs into Epics and User Stories and push them directly to a PM's active sprint.

Could Have (Future Roadmap)

- Native Figma/Penpot API synchronization for automatic UI generation in design tools.
- Voice-to-text integration for real-time meeting transcription directly within the studio.

Won't Have (Excluded from Current Scope)

- Real-time Zoom/Teams audio/video stream ingestion.
- Automatic deployment of code directly to production servers.

3.0 Stakeholder Requirements & Personas

3.1 Primary Personas

Persona A: The Senior Business Analyst (SBA)

Description: The primary end-user of the platform. They are responsible for gathering requirements from clients and translating them into technical specifications.

Pain Points: Spending 80% of their week formatting Word documents and manually drawing UML diagrams in Visio.

Requirements: Requires a conversational interface to upload meeting minutes, validate business logic, and instantly generate structured deliverables to present to clients. The output must be perfectly formatted and immediately exportable as a PDF.

Persona B: The Project Manager (PM)

Description: Responsible for tracking project velocity and assigning tasks to developers based on the BA's requirements.

Pain Points: Manually copying requirements from a BRD into Jira tickets, often losing context or formatting in the process.

Requirements: Requires the "Jira Sync" functionality to automatically parse the FRD into a hierarchy of Epics, User Stories, and Tasks, and push it directly into their active sprint board via API.

Persona C: The Quality Assurance Lead (QA)

Description: Responsible for ensuring the final software product meets the business requirements.

Pain Points: Waiting until development is completely finished before they can begin writing and executing test scripts.

Requirements: Requires the "E2E Automation Suite" feature to automatically generate Playwright automation scripts based purely on the AI-generated UI prototypes, allowing them to initiate Shift-Left testing before developers write a single line of backend code.

3.2 Secondary Personas (Governance & Security)

Persona D: Chief Information Security Officer (CISO)

Description: Responsible for ensuring the enterprise does not violate HIPAA, GDPR, or SOC2 compliance laws by leaking data to public LLMs.

Pain Points: Employees using unauthorized shadow-IT AI tools.

Requirements: Requires absolute cryptographic proof that the "AI Shield" is actively intercepting and scrubbing all PII data before network transmission. Requires the Immutable Audit Log to track exactly which user generated which document at what timestamp.

Persona E: The Enterprise Administrator

Description: Manages billing and corporate IT access.

Pain Points: Paying for AI API tokens consumed by unauthorized users or accidental usage spikes.

Requirements: Requires an Admin Dashboard to monitor API usage, manage user RBAC, and view global organization metrics.

4.0 System Architecture & Database Schema

4.1 Architectural Overview

The BA Studio is built on a highly scalable, serverless architecture optimized for AI streaming and real-time UI updates.

- **Frontend Framework:** Next.js 16 (App Router) utilizing React Server Components (RSC) for optimized initial load times.
- **Styling:** Tailwind CSS combined with Framer Motion for micro-animations and UI interactions.
- **Backend Execution:** Vercel Serverless Edge Functions configured with a maximum execution duration of 60 seconds to accommodate long-running LLM generation tasks.
- **Database:** PostgreSQL hosted on Neon (Serverless Postgres), interfacing with the application via Prisma ORM.
- **AI Engine:** Google DeepMind Gemini API (`gemini-2.5-pro` for deep reasoning tasks, `gemini-2.5-flash` for high-speed synchronization tasks).

4.2 Database Schema (Prisma Data Models)

To support Multi-Tenant RBAC and audit requirements, the database schema is strictly normalized. The core models include:

Organization Model

Represents a corporate tenant. Controls billing limits and subscription status.

```
model Organization {
```

```

id          String    @id @default(cuid())
name        String
plan        String    @default("ENTERPRISE")
isActive    Boolean    @default(true)
maxUsers    Int        @default(50)
createdAt   DateTime   @default(now())
updatedAt   DateTime   @updatedAt

members     OrganizationMember[]
projects    Project[]
auditLogs   AuditLog[]
}

```

User & OrganizationMember Models

Implements strict Role-Based Access Control connecting a user to a tenant with a specific role (`ADMIN`, `MEMBER`, or `VIEWER`).

```

model OrganizationMember {
  id          String      @id @default(cuid())
  userId       String
  organizationId String
  role         Role        @default(VIEWER) // Enums: ADMIN, MEMBER,
VIEWER
  joinedAt     DateTime    @default(now())

  user         User        @relation(fields: [userId], references: [id],
onDelete: Cascade)
  organization Organization @relation(fields: [organizationId],
references: [id], onDelete: Cascade)

  @@unique([userId, organizationId])
}

```

AuditLog Model

Maintains SOC2 compliance by permanently recording all critical actions. These records cannot be modified or deleted via the application UI.

```
model AuditLog {
  id          String      @id @default(cuid())
  organizationId String
  userId      String
  userEmail   String
  action      String      // e.g., "LOGIN", "DOCUMENT_GENERATED"
  resourceType String?    // e.g., "FRD", "UML Diagram"
  metadata    Json?
  timestamp   DateTime    @default(now())

  organization Organization @relation(fields: [organizationId],
references: [id], onDelete: Cascade)
}
```

5.0 Detailed Functional Requirements (FR)

5.1 Sub-System A: The Analysis Brain (Intake Engine)

The Intake Engine is responsible for parsing unstructured text from the user, determining business feasibility, and establishing the foundational "Snapshot" of confirmed requirements.

ID	Title	Detailed Description	Acceptance Criteria
FR-01	MOM Parsing & Domain Detection	The system must ingest raw Meeting Minutes (MOM) via the <code>/api/analyze`</code> route. The AI must extract the primary industry domain (e.g., Healthcare, FinTech). If multiple unrelated domains are detected (e.g., Aviation and Retail banking), it must flag a "Multi-Domain Conflict."	- API must return a JSON object with <code>`domainDetected`</code> string. - API must return <code>`multiDomainDetected`</code> boolean flag.

FR-02	Readiness Checklist & Scoring	The AI must objectively evaluate the user's input against a 7-point Readiness Checklist (Domain Feasibility, Stakeholder Clarity, Core Process, Success Criteria, Scope Boundary, Integration Points, Non-Functional Needs). It must assign a `readinessScore` (0-10).	• The system must set `sessionState` to "QUESTIONING" if the score is < 4. • The system must deny document generation until the state is "READY".
FR-03	Feasibility Gatekeeper	The AI must scan the input for physical/logical flaws (e.g., "a boat that drives on the highway") or internal contradictions (e.g., "offline-first app with real-time server sync").	• Any detected flaws must be populated into the `feasibilityIssues` array. • The UI must display these flaws to the user as critical blockers.

5.2 Sub-System B: The Multi-Agent Generation Engine

Once requirements are validated, the system utilizes specialized AI Agents, each configured with extreme systemic constraints, to generate specific artifacts via the `/api/chat` route.

ID	Title	Detailed Description	Acceptance Criteria
FR-04	BRD Generation (Senior BA Agent)	Generates a comprehensive Business Requirements Document. The prompt forces strict adherence to MoSCoW prioritization for feature mapping.	- Output must include an Executive Summary. - Output must prioritize features into Must/Should/Could/Won't Have.
FR-05	FRD Generation (Senior BA Agent)	Generates the Functional Requirements Document. The system is hard-coded to require a Requirement Traceability Matrix (RTM) at the bottom of the document by default.	- Output must utilize strict FR-XXX numbering format. - Output must include 100% plain-text Acceptance Criteria for every requirement. - Output must end with an RTM table mapping FRs to business objectives.
FR-06	SRD Generation (Systems	Generates the System Requirements Document focusing	Output must explicitly adhere to ISO/IEC 25010 standards

	Analyst Agent)	on infrastructure, data flow, and non-functional bounds.	for software quality metrics (Reliability, Usability, Security, etc.).
FR-07	Regulatory Advisor Agent	Analyzes the confirmed requirements for sovereign compliance gaps (e.g., identifying the need for GDPR compliance if EU data is mentioned).	• Output must format as a clean, plain-text regulatory audit report.

5.3 Sub-System C: Visual Architecture Engine

This sub-system is responsible for generating visual and interactive artifacts. Because LLMs are prone to syntax errors when writing diagram code, this engine uses highly hardened "Nuclear Syntax Constraints".

ID	Title	Detailed Description	Acceptance Criteria
FR-08	UML Generation (System Architect Agent)	Generates a PlantUML Class Diagram visualizing data models and actors. The prompt strictly bans parentheses or spaces in relationship labels to prevent rendering crashes.	- Output must use <code>`plantuml`</code> code fences. - Output is limited to a maximum of 12 classes to ensure absolute stability in the rendering pipeline.
FR-09	Flowchart Generation (Process Architect Agent)	Generates a Mermaid.js Process Map. The LLM is strictly instructed to use only <code>`graph TD`</code> , solid arrows (<code>`-->`</code>), and to place all text strictly inside quoted nodes (e.g., <code>`A["Action"]`</code>).	- Output must be strictly parsable Mermaid syntax. - The server must run a RegEx sanitizer (<code>`replace(/\ /?\s*-\+\s*-\>/g, ' --> ')`</code>) before sending the stream to the client to fix any malformed arrows the AI hallucinates.
FR-10	Rapid Prototyping (UI/UX Agent)	Generates a high-fidelity, interactive SaaS dashboard prototype. The LLM outputs a single, self-contained	Output must utilize a deep navy/blue gradient visual aesthetic (Glassmorphism).

		HTML snippet utilizing Tailwind CSS for styling and Alpine.js for interactive state (e.g., clicking tabs).	• Output must NOT include ```, ```, or ``` tags (must be an embeddable snippet). • Output must include functional `@click` handlers for navigation.
FR-11	Wireframe Generation (UX Architect Agent)	Generates a low-fidelity grayscale wireframe to establish layout consensus before high-fidelity prototyping.	• Output must maintain "Structural Symmetry" with the functional requirements (exact screen names/IDs). • Output must use actual text labels, banning "Lorem Ipsum" placeholder text.

5.4 Sub-System D: Developer Automation & Operations

This sub-system bridges the gap between Business Analysis and Engineering by converting requirements directly into code and tickets.

ID	Title	Detailed Description	Acceptance Criteria
FR-12	E2E Automation Suite Generation	The system sends the generated Prototype code and the AI-generated Test Cases to the <code>`/api/generate/tests`</code> route. The LLM writes a robust Playwright automation script to test the UI.	- The system must fall back to using Wireframes or Text-based Test Cases if the Prototype has not been generated yet. - The API route must utilize a <code>`maxDuration = 60`</code> configuration to prevent server timeouts while the AI writes the complex code.
FR-13	Infrastructure as Code (IaC)	The system generates Terraform HCL manifests based on the System Requirements (e.g., defining AWS RDS instances if a database is required).	The API must utilize a dynamic Regex <code>(`""\w*\s*([s\S]*?)\s*"/`)</code> to safely extract the code block from the AI's response, regardless of whether it labels the block as

			<code>`typescript`,</code> <code>`terraform`, or `hcl`.</code>
FR-14	Jira Synchronization	The system pushes the FRD data directly to Jira. The LLM formats the requirements into a specific Epic > Story > Task JSON manifest which the backend API then POSTs to the Atlassian API.	• System must establish an OAuth 2.0 or Basic Auth connection using the credentials provided in the user's Settings modal. • Upon success, the UI must display a confirmation toast.

5.5 Sub-System E: Sovereign Versioning & UI State

This ensures the user experience is fluid, states are persisted, and document regeneration is easily accessible.

ID	Title	Detailed Description	Acceptance Criteria
FR-15	Sovereign Versioning Engine (Impact Analysis)	Every time the user updates requirements via the Chat, the <code>/api/analyze`</code> route returns a cumulative <code>`snapshot`</code> of requirements. The system calculates the delta between the old snapshot and the new snapshot to determine which documents are now "Stale".	• The state variable <code>`scopeHistory`</code> must append the new snapshot object. • If <code>`scopeHistory.length > 1`</code>, impacted documents must display a warning icon indicating they need review.

FR-16	Permanent Regenerate Functionality	The Document Toolbar must permanently display a "Regenerate" button to allow users to force the AI to re-write a document.	• If no revisions have been made (<code>scopeHistory.length === 1`</code>), the button must read "Regenerate Document".</li> <li>• If revisions have been made (<code>scopeHistory.length > 1`</code>), the button text must dynamically update to "Regenerate with v{X} Updates".
FR-17	Universal Top Bar Toolbar	The Top Bar provides global export functionalities (Save Session, Export All PDF) and context-aware tools (Edit, Jira Sync, Copy).	• The Top Bar buttons must remain visible but enter a <code>`disabled`</code> state (<code>`opacity-40`</code>) while the AI is actively generating a document, preventing the UI layout from shifting or disappearing.

6.0 Non-Functional Requirements (NFR)

6.1 Security & Compliance

- **NFR-01 (PII Redaction / AI Shield):** The ``src/lib/pii.ts`` module MUST be invoked on all user message payloads before transmission to the Gemini API. It must utilize RegEx matching to redact all Emails, Phone Numbers, Social Security Numbers (SSN), IP Addresses, and Credit Card numbers, replacing them with standardized tags (e.g., ``[REDACTED_EMAIL]``). This is mandatory for enterprise data safety protocols.
- **NFR-02 (SOC2 Audit Trails):** Every login event and document generation event MUST be recorded in the PostgreSQL ``AuditLog`` table with an ISO 8601 timestamp, the exact ``userId``, the ``organizationId``, and the resource type accessed. This log must be immutable via the application interface.
- **NFR-03 (NextAuth RBAC):** Users possessing a ``VIEWER`` role MUST be programmatically blocked at the API route level (``/api/chat``, ``/api/analyze``, ``/api/generate/tests``) from executing LLM queries to protect corporate billing quotas. They may only view existing saved documents.

6.2 Performance & Scalability

- **NFR-04 (API Rate Limiting):** The ``src/lib/rate-limit.ts`` module MUST implement an in-memory Rate Limiter restricting users to a maximum of 10 generation requests per minute per IP/User identifier to mitigate DDoS attacks and API quota exhaustion.
- **NFR-05 (Serverless Edge Configuration):** All API Routes handling complex document or code generation MUST utilize the ``export const maxDuration = 60;`` configuration variable to override the Vercel default 10-second timeout, preventing 504 Gateway Timeouts during extensive AI processing tasks.

- **NFR-06 (Streaming Responses):** The `/api/chat` and `/api/analyze` routes MUST utilize `ReadableStream` architecture to stream tokens to the frontend in real-time, providing immediate visual feedback to the user and reducing perceived latency to under 2 seconds.

7.0 Requirement Traceability Matrix (RTM)

This matrix maps all core functional requirements to their corresponding business objectives to ensure alignment and verify production readiness.

Requirement ID	Requirement Title	Business Objective / Value	Validation Status
FR-01	MOM Parsing & Domain Detection	Automate initial requirements gathering and establish context.	Verified in Production
FR-02	Readiness Checklist & Scoring	Prevent garbage-in, garbage-out data generation.	Verified in Production
FR-03	Feasibility Gatekeeper	Identify logical flaws before engineering begins.	Verified in Production
FR-04	BRD Generation	Reduce Business Analysis latency by 80%.	Verified in Production
FR-05	FRD Generation (with RTM)	Provide structured rules for engineering teams.	Verified in Production
FR-06	SRD Generation	Establish system performance and security benchmarks.	Verified in Production
FR-07	Regulatory Advisor Agent	Mitigate legal compliance risks early in the lifecycle.	Verified in Production

FR-08	UML Generation	Visualize system architecture for backend developers.	Verified in Production
FR-09	Flowchart Generation	Visualize user journeys for stakeholders.	Verified in Production
FR-10	UI Prototypes (Alpine/Tailwind)	Provide tangible, clickable assets to secure stakeholder sign-off rapidly.	Verified in Production
FR-11	Wireframe Generation	Establish basic layout consensus.	Verified in Production
FR-12	E2E Automation Suite Generation	Shift Quality Assurance testing left, reducing QA bottleneck.	Verified in Production
FR-13	Infrastructure as Code (IaC)	Automate DevOps provisioning based on requirements.	Verified in Production
FR-14	Jira Synchronization	Automate Project Management overhead and sprint planning.	Verified in Production
FR-15	Sovereign Versioning Engine	Maintain absolute consistency when scope creep occurs.	Verified in Production
FR-16	Permanent Regenerate Functionality	Improve UX and allow manual correction of AI hallucinations.	Verified in Production

NFR-01	PII Redaction (AI Shield)	Guarantee Enterprise Data Security and HIPAA/PCI compliance.	Verified in Production
NFR-02	SOC2 Audit Trails	Provide irrefutable proof of system usage for auditors.	Verified in Production
NFR-03	NextAuth RBAC	Protect corporate IP and prevent unauthorized billing usage.	Verified in Production
NFR-04	API Rate Limiting	Protect system stability and financial resources.	Verified in Production
NFR-05	Serverless Edge Configuration	Prevent pipeline failures during massive text generation.	Verified in Production

Sign-off Note: This document represents the finalized, exhaustive technical requirements for the Gaurav Nandhan BA Studio Enterprise Edition v1.0.0. All requirements listed above have been audited, implemented, and verified in the production environment.